Chapter 8: Multidimensional Arrays

COMP 110
Two-dimensional arrays
Examples, discussion, and practice

Learning goals

Represent tables with row and column indexes
Create rectangular and ragged arrays
Traverse, summarize, and shuffle a grid
Pass two-dimensional arrays to methods
Explain shared rows and independent copies

Discussion: choosing a grid

A distance lookup needs a starting city and a destination city.

A classroom score table needs a student and an assignment.

What should each index represent?

Example distance table

Miles	Chicago	Boston	New York	Atlanta	Miami	Dallas	Houston
Chicago	0	983	787	714	1375	967	1087
Boston	983	0	214	1102	1763	1723	1842
New York	787	214	0	888	1549	1548	1627
Atlanta	714	1102	888	0	661	781	810
Miami	1375	1763	1549	661	0	1426	1187
Dallas	967	1723	1548	781	1426	0	239
Houston	1087	1842	1627	810	1187	239	0

Row = starting city. Column = destination city.

Looking up a distance with two indexes

```
int[][] distances = {
    {0, 983, 787, 714, 1375, 967, 1087},
    {983, 0, 214, 1102, 1763, 1723, 1842},
    {787, 214, 0, 888, 1549, 1548, 1627},
    {714, 1102, 888, 0, 661, 781, 810},
    {1375, 1763, 1549, 661, 0, 1426, 1187},
    {967, 1723, 1548, 781, 1426, 0, 239},
    {1087, 1842, 1627, 810, 1187, 239, 0}
};
System.out.println(distances[0][1]);
System.out.println(distances[1][0]);
```

distances[0][1] uses Chicago's row and Boston's column.

Declaration, creation, and access

```
int[][] matrix;
matrix = new int[3][4];
matrix[1][2] = 7;
System.out.println(matrix.length);
System.out.println(matrix[1].length);
System.out.println(matrix[1][2]);
System.out.println(matrix[0][0]);
```

new int[3][4] creates 3 rows with 4 int elements each.

Two-dimensional array initializers

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};
System.out.println(Arrays.deepToString(matrix));
matrix = new int[][]{{7, 8}, {9, 10}};
System.out.println(matrix[1][0]);
```

Each inner initializer supplies one row.

Outer length and row lengths

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9},
    {10, 11, 12}
};
System.out.println(matrix.length);
System.out.println(matrix[0].length);
System.out.println(matrix[3][2]);
// System.out.println(matrix[4].length); // Invalid row.
```

matrix.length counts rows. matrix[row].length counts that row's elements.

Ragged arrays have different row lengths

```
int[][] rows = {
    {1, 2, 3, 4, 5},
    {2, 3, 4, 5},
    {3, 4, 5},
    {4, 5},
    {5}
};
for (int r = 0; r < rows.length; r++) {
    System.out.println(r + ": " + rows[r].length);
}
```

Java permits each row to be a separate array of a different length.

Creating the row arrays separately

```
int[][] rows = new int[3][];
System.out.println(rows[0] == null);
rows[0] = new int[]{10, 20};
rows[1] = new int[0];
rows[2] = new int[]{30};
System.out.println(rows[0].length);
System.out.println(rows[1].length);
System.out.println(rows[2][0]);
```

new int[3][] creates three null row references.

Quiz 1: dimensions and bounds

Given new int[2][3]:
1. What are the outer and row lengths?
2. Is [1][2] a valid element?
3. Is [2][0] a valid element?

After new int[2][], what is the first row?

Nested loops visit one row at a time

```
static void printMatrix(int[][] matrix) {
    for (int r = 0; r < matrix.length; r++) {
        for (int c = 0; c < matrix[r].length; c++) {
            System.out.print(matrix[r][c] + " ");
        }
        System.out.println();
    }
}
```

The inner bound uses the current row's length.

Nested for-each loops

```
int[][] matrix = {{1, 2}, {3}};
for (int[] row : matrix) {
    for (int value : row) {
        System.out.println(value);
    }
}
```

The outer variable is an int[] row. The inner variable is an int value.

Discussion: choosing loop bounds

One row has length 3 and another length 1.

What happens if every inner loop uses the first row's length?

When do we need explicit row and column indexes rather than for-each?

Trace: a value from each coordinate

```
int[][] matrix = new int[2][3];
for (int r = 0; r < matrix.length; r++) {
    for (int c = 0; c < matrix[r].length; c++) {
        matrix[r][c] = 10 * r + c;
    }
}
System.out.println(Arrays.deepToString(matrix));
```

Prediction: write both rows before running the program.

Trace results

r = 0: write 0, 1, and 2
r = 1: write 10, 11, and 12

Final array:
[[0, 1, 2], [10, 11, 12]]

The inner loop starts at 0 for each row.

Filling a matrix from input

```
Scanner input = new Scanner(System.in);
int[][] matrix = new int[2][3];
for (int r = 0; r < matrix.length; r++) {
    for (int c = 0; c < matrix[r].length; c++) {
        matrix[r][c] = input.nextInt();
    }
}
System.out.println(Arrays.deepToString(matrix));
```

Input 1 2 3 4 5 6 fills the first row before the second.

Filling a matrix with random values

```
Random random = new Random(8);
int[][] matrix = new int[2][3];
for (int r = 0; r < matrix.length; r++) {
    for (int c = 0; c < matrix[r].length; c++) {
        matrix[r][c] = random.nextInt(10);
    }
}
System.out.println(Arrays.deepToString(matrix));
```

nextInt(10) gives an int from 0 through 9.

Summing all elements

```
static long total(int[][] matrix) {
    long sum = 0;
    for (int[] row : matrix) {
        for (int value : row) sum += value;
    }
    return sum;
}
```

A long accumulator reduces the risk of overflowing an int total.

A separate total for each row

```
static long[] rowTotals(int[][] matrix) {
    long[] totals = new long[matrix.length];
    for (int r = 0; r < matrix.length; r++) {
        for (int value : matrix[r]) {
            totals[r] += value;
        }
    }
    return totals;
}
```

For {{1, 2, 3}, {4, 5}}, the row totals are [6, 9].

A separate total for each column

```
static long[] columnTotals(int[][] matrix) {
    int columns = matrix.length == 0 ? 0 : matrix[0].length;
    long[] totals = new long[columns];
    for (int[] row : matrix) {
        for (int c = 0; c < columns; c++) {
            totals[c] += row[c];
        }
    }
    return totals;
}
```

For {{1, 2, 3}, {4, 5, 6}}, column totals are [5, 7, 9].

Quiz 2: row and column totals

For [[1, 2, 3], [4, 5, 6]]:

1. What is the total of all elements?
2. What are the row totals?
3. What are the column totals?
4. Why does our column method require equal row lengths?

The first row with the largest total

```
static int largestRowTotal(int[][] matrix) {
    long[] totals = RowTotalsDemo.rowTotals(matrix);
    if (totals.length == 0) return -1;
    int best = 0;
    for (int r = 1; r < totals.length; r++) {
        if (totals[r] > totals[best]) best = r;
    }
    return best;
}
```

Use > to retain the first row when totals tie.

The first position of the maximum

```
static int[] maximumPosition(int[][] matrix) {
    int bestRow = -1, bestColumn = -1;
    for (int r = 0; r < matrix.length; r++) {
        for (int c = 0; c < matrix[r].length; c++) {
            if (bestRow == -1 ||
                matrix[r][c] > matrix[bestRow][bestColumn]) {
                bestRow = r;
                bestColumn = c;
            }
        }
    }
    return new int[]{bestRow, bestColumn};
}
```

Visit rows in order, then columns. Use > so the first tie wins.

Finding the minimum value

```
static int minimum(int[][] matrix) {
    int best = Integer.MAX_VALUE;
    for (int[] row : matrix) {
        for (int value : row) {
            if (value < best) best = value;
        }
    }
    return best;
}
```

Precondition: at least one element exists.

Discussion: contracts and ties

Should an empty matrix return a sentinel
or cause a method to reject the input?

Should equal maxima choose the first
or the last position?

What does “first” mean for a ragged array?

Shuffling all cells of a rectangle

```
static void shuffle(int[][] matrix, Random random) {
    if (matrix.length == 0) return;
    int columns = matrix[0].length;
    if (columns == 0) return;
    int cells = matrix.length * columns;
    for (int i = cells - 1; i > 0; i--) {
        swap(matrix, columns, i, random.nextInt(i + 1));
    }
}
```

Treat the cells as one sequence, then apply Fisher-Yates.

Flat indexes and two-dimensional positions

```
static void swap(int[][] matrix, int columns, int i, int j) {
    int r1 = i / columns, c1 = i % columns;
    int r2 = j / columns, c2 = j % columns;
    int temp = matrix[r1][c1];
    matrix[r1][c1] = matrix[r2][c2];
    matrix[r2][c2] = temp;
}
```

row = index / columns. column = index % columns.

Quiz 3: positions and shuffling

For a rectangle with 3 columns:

1. Which coordinate matches flat index 5?
2. Which flat index matches [1][1]?
3. Does shuffling preserve all values?
4. Can the same / and % mapping
   handle ragged rows?

Passing a matrix: changing an element

```
int[][] matrix = {{1, 2}, {3, 4}};
changeCell(matrix);
System.out.println(Arrays.deepToString(matrix));

static void changeCell(int[][] matrix) {
    matrix[0][0] = 99;
}
```

The method receives a copy of the outer array reference.

Passing a matrix: replacing one row

```
int[][] matrix = {{1, 2}, {3, 4}};
replaceFirstRow(matrix);
System.out.println(Arrays.deepToString(matrix));

static void replaceFirstRow(int[][] matrix) {
    matrix[0] = new int[]{7, 8, 9};
}
```

matrix[0] is an element of the shared outer array.

Passing a matrix: reassigning the parameter

```
int[][] matrix = {{1, 2}, {3, 4}};
replaceMatrix(matrix);
System.out.println(Arrays.deepToString(matrix));

static void replaceMatrix(int[][] matrix) {
    matrix = new int[][]{{9, 9}};
}
```

Reassigning the local parameter leaves the caller reference unchanged.

Copying the outer array still shares rows

```
int[][] original = {{1, 2}, {3, 4}};
int[][] copy = original.clone();
copy[0][0] = 99;
System.out.println(original == copy);
System.out.println(original[0] == copy[0]);
System.out.println(Arrays.deepToString(original));
```

clone creates a new outer array and copies the row references.

Copying each int row independently

```
static int[][] copyRows(int[][] matrix) {
    int[][] copy = new int[matrix.length][];
    for (int r = 0; r < matrix.length; r++) {
        copy[r] = matrix[r].clone();
    }
    return copy;
}
```

Allocate a new outer array, then clone every primitive row.

Quiz 4: matrix parameters and copies

1. Can changing matrix[0][0] affect the caller?
2. Can assigning matrix[0] = new int[]{7}
   affect the caller?
3. Can assigning the local matrix parameter
   replace the caller's reference?
4. Does cloning only the outer array
   make independent int rows?

Checking a rectangular-shape contract

```
static boolean isRectangular(int[][] matrix) {
    if (matrix == null) return false;
    if (matrix.length == 0) return true;
    if (matrix[0] == null) return false;
    int columns = matrix[0].length;
    for (int[] row : matrix) {
        if (row == null || row.length != columns) return false;
    }
    return true;
}
```

This policy accepts empty input and rejects null rows.

Extension: transposing a rectangle

```
static int[][] transpose(int[][] matrix) {
    int rows = matrix.length;
    int columns = rows == 0 ? 0 : matrix[0].length;
    int[][] result = new int[columns][rows];
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < columns; c++) {
            result[c][r] = matrix[r][c];
        }
    }
    return result;
}
```

The result exchanges row and column positions.

Discussion: selecting a data shape

A course has the same assignments for everyone.
A survey allows a different number of answers per person.

Which task naturally fits a rectangle?
Which task naturally fits ragged rows?
What should null or missing data mean?

Exit exercise: counting matching cells

Design countAbove(int[][] matrix, int limit).
Count elements strictly greater than limit.
Support ragged rows and empty rows.

Predict the result for {{2, 8}, {}, {6}}
and limit 5.

Exit exercise: one possible solution

```
static long countAbove(int[][] matrix, int limit) {
    long count = 0;
    for (int[] row : matrix) {
        for (int value : row) {
            if (value > limit) count++;
        }
    }
    return count;
}
```

{{2, 8}, {}, {6}} with limit 5 gives 2. Empty input gives 0.

Chapter review

A two-dimensional array stores row references.
Each row has its own length.
Nested loops can use each row's bounds.
Rectangular operations need a shape contract.
Method calls copy reference values.
Independent int rows require row-by-row copies.